



DATA STRUCTURES

CONTENTS



20. Tree ADT – II

TV. GEETHA



Welcome to the e-PG Pathshala Lecture Series on Data Structures. We have understood the basic concept of an important ADT – the Tree ADT. In this module we will discuss one very important type of tree ADT that is binary tree ADT.

Learning Objectives

The learning objectives of the module are as follows:

- To compare General Trees and Binary Trees
- To discuss Binary Trees with some examples
- To describe some operations of Binary Trees
- To discuss the various representations of binary Trees
- To explain the various Binary Tree Traversals

20.1 Introduction

As we have discussed in the previous module a general tree is a tree that can have any number of children. Here a node r , is a specially designated node called the root and sets that are themselves general trees, which are subtrees of r . A binary tree on the other hand is a set T of nodes such that either T is empty, or T is partitioned into 3 disjoint subsets- the node r called the root and two possibly empty sets that are binary trees, called left and right subtrees of r .

20.1.1 Binary Trees

Let us now define a binary tree. A binary tree is an ordered tree (Figure 20.1) with the following properties:

1. Every node has at most two children
2. Each child node is labeled as being either a left child or a right child.
3. A left child precedes a right child in the ordering of children of a node, (Children form an ordered pair).

Nodes with no successors are called leaves. The roots of the left and right subtrees of a node " i " are called the "children of i "; the node i is their parent; the subtrees are siblings. A child always has one parent. In a binary tree, a parent has at most two children. The depth of a node in the binary tree is the number of edges from the root to the specific node. The height of a node is the number of edges from the node to the deepest leaf of the tree. While depth of a node is measured in terms of distance from the root node, height of a node is measured in terms of distance to the deepest leaf. The depth of a binary tree is the depth of its deepest node. The height of a tree is the height of the root.

20.1.2 Maximum Number of Nodes in a Binary Tree

The maximum number of nodes at depth i of a binary tree is 2^i , The number of internal nodes is at least h and at most $2^h - 1$. The total number of nodes in a binary tree is at least $2^h + 1$ and at most $2^{h+1} - 1$ where h is height of the binary tree. This can be proved by induction as follows where $i \geq 0$.

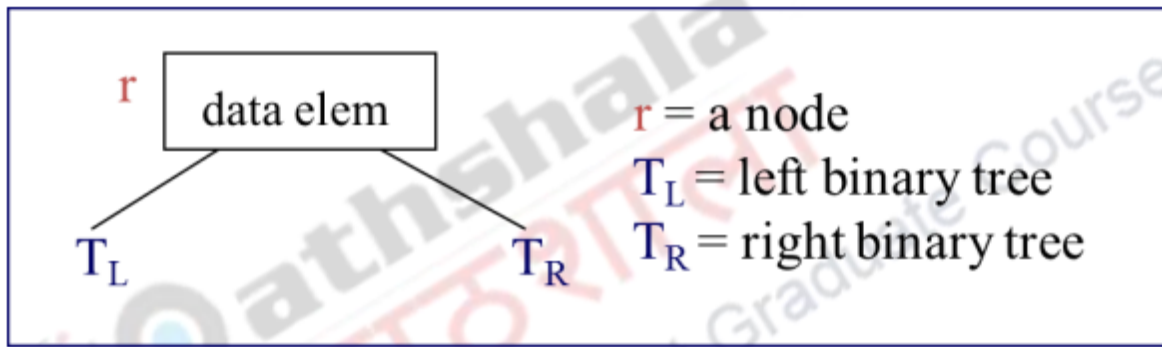


Figure 20.1 Nodes of a Binary Tree

20.1.3 Binary Tree ADT

The Binary Tree ADT is an ordered tree with the following properties

1. Each node can have at most two subtrees
2. Each subtree is identified as being either left subtree or the right subtree of its parent
3. It may be empty

Note:

- Property 1 ensures that each node can have maximum two children
- The order between the children of a node is specified by labeling its children as left child and right child

20.2 Examples of Binary Tree

Example 1: Expression Tree

The expression tree (Figure 15.2) is the central data structure in compiler design. Interior nodes consists of operators and the leaf nodes have operands. An expression is evaluated by applying the operator present at the root to the values obtained by recursively evaluating the left and right subtrees.

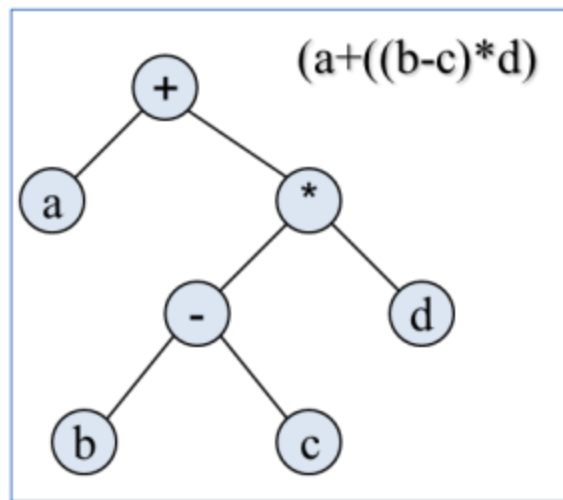


Figure 20.2 Expression Tree

Example 2: Huffman Coding

Huffman Coding Tree (Figure 20.3) is used in a simple but effective data compression algorithm. In this tree, each symbol is stored at a leaf node. To generate the code of a symbol, we traverse the tree from the root to the leaf node that contains the symbol such that each left link corresponds to 0 and each right link corresponds to 1.

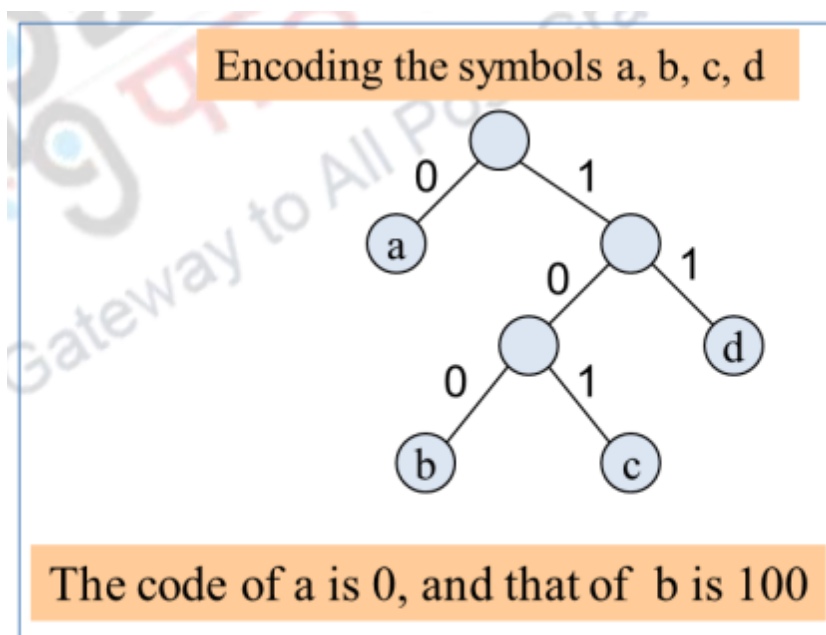


Figure 20.3 Tree for Huffman Encoding

20.3 Special Types of Binary Trees

20.3.1 Full Binary Tree

In a Full Binary Tree, all nodes in the tree have exactly 0 or 2 children. Every node has exactly two children at all levels, except the last level. Nodes in last level have 0 children. Each node has left and right subtrees of the same height or depth. Figure 20.4 shows the level and the number of nodes at each level.

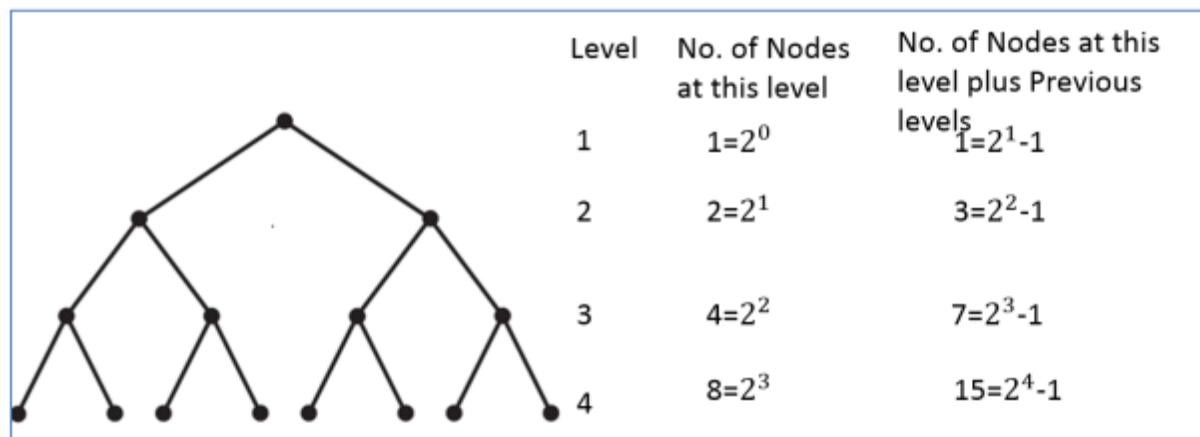


Figure 20.4 Full Binary Tree

Complete and Balanced Binary Trees

A binary tree T with n levels is complete if all levels except possibly the last are completely full, and the last level has all its nodes to the left side. A Complete Binary Tree is a binary tree similar to a full binary tree down to level h-1, with level h filled in from left to right.

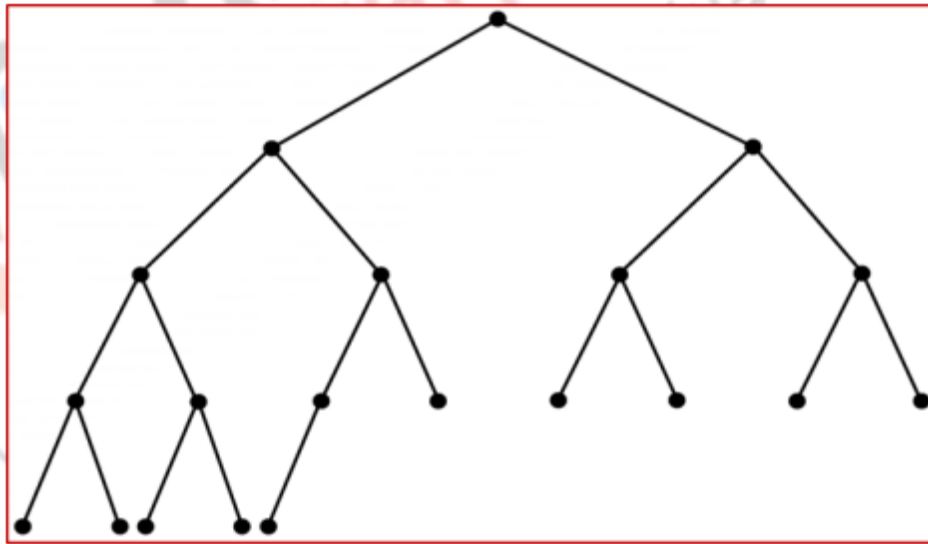


Figure 20.5 Complete Binary Tree

In a Balanced binary tree, the height of any node's right subtree differs from the height of the node's left subtree by no more than 1. Naturally a complete binary tree is balanced.

20.4 Operations – Binary Tree ADT

The operations of the Binary Tree ADT can be classified as general and special accessor operations, generic operations, query operations and update operations. Here positions are used to abstract nodes.

20.4.1 Generic Accessor Operations

- **root()**: Return the position of the tree's root; an error occurs if the tree is empty.
- **parent(p)**: Return the position of the parent of p; an error occurs if p is the root.
- **children(p)**: Returns an iterable collection containing the children of node p stores the left child of p before the right child of p.

20.4.2 Specific Accessor Operations:

- **position left(p)**: returns the left child of p, an error condition occurs if p has no left child.
- **position right(p)**: returns the right child of p, an error condition occurs if p has

no right child.

- **boolean hasLeft(p)**: tests whether p has a left child
- **boolean hasRight(p)**: tests whether p has a right child

20.4.3 Generic Operations

These operations are similar to those we have seen up to now and are generic and applicable across many types of ADTs.

- **size()**: returns the number of nodes in the tree.
- **isEmpty()**: Tests whether the tree has any nodes or not.
- **Iterator()**: Returns an iterator of all the elements stored at nodes of the tree.
- **positions()**: Returns an iterable collection of all the nodes of the tree.

20.4.4 Query Operations

Tree ADT supports the following Boolean query operations:

- **isInternal(p)**: Test whether node p is an internal node
- **isExternal(p)**: Test whether node p is an external node
- **isRoot(p)**: Test whether node p is the root node

20.4.5 Update Operations

The Tree ADT – supports the following update operation:

- **replace(p, e)**: Replace with e and return the element stored at node p.

20.5 Array Representation of Binary Tree ADT

One of the ways of representing the binary tree using arrays is by storing the nodes in an array based on level numbering of the nodes of the tree T. Some locations are skipped through the numbering process, e.g. numbers 8 & 9 (Figure 20.6). This is a simple and efficient implementation. The operations of finding root, parent, left, right, hasLeft, hasRight, isInternal, isExternal, and isRoot by using

simple arithmetic operations on the numbers, associated with each node *v* involved in the operation:

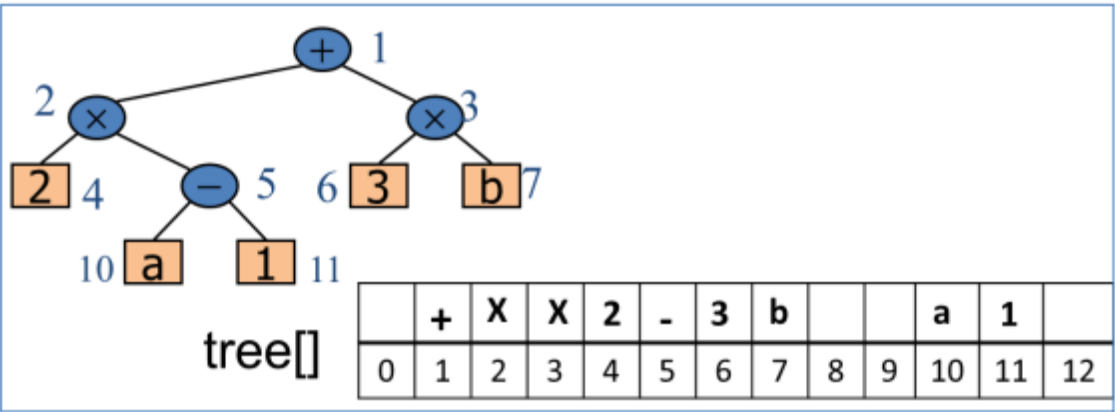


Figure 20.6 Level based Array Representation of Binary Tree

For each node *v* of *T*, let *p(v)* be an integer defined as follows:

- If node *v* is the root, then $p(v) = 1$
- if node *v* is the left child of node *u*, then $p(v) = 2 * p(u)$
- if node *v* is the right child of node *u*, then $p(v) = 2 * p(u) + 1$

The binary tree is implemented by means of an extendable array. **pM** be the maximum value of *p(v)* over all nodes of *T*. Then the size of array list *S* is $N = pM + 1$, since index 0 of *S* has no nodes stored in it. In worst case, *S* has $N = 2^d$ elements where *d* is the height of the tree. The time complexity associated with some of the Operations is given in Table 20.1 However this implementation is effective when the binary tree is complete or full and the operations such as insertion of a node and deletion of a node is not needed.

<i>Operations</i>	<i>Time</i>
<i>size, isEmpty</i>	<i>O(1)</i>
<i>iterator, positions</i>	<i>O(n)</i>
<i>replace</i>	<i>O(1)</i>
<i>root, parent, children, left, right</i>	<i>O(1)</i>
<i>hasLeft, hasRight, isInternal, isExternal, isRoot</i>	<i>O(1)</i>

Table 20.1 Time Complexity using Level based Array Implementation

20.6 Linked Representation for Binary Tree ADT

In this representation of a binary tree, a node consists of the data element, pointer to the parent node, pointer to the left child node and pointer to the right child node (Figure 20.7). Position ADT is used to represent the node. There are other Linked representations of Binary tree which does not include a pointer to the parent node.

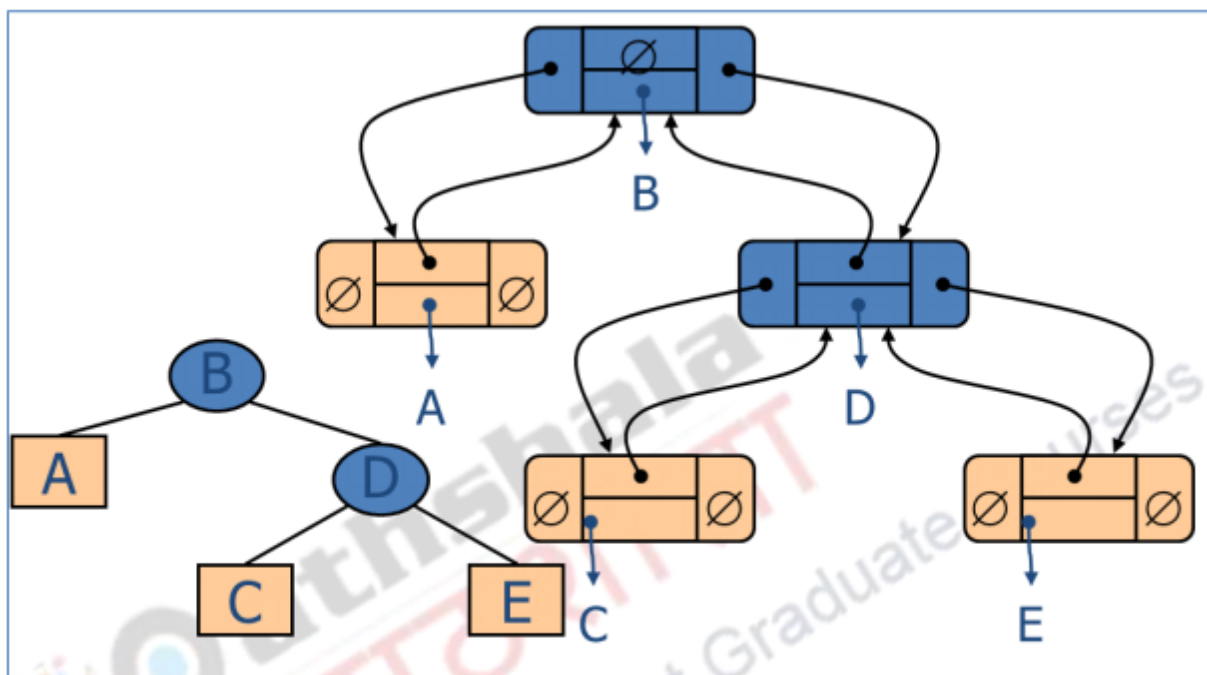


Figure 20.7 Linked List Representation of Binary Tree

For the Linked Representation of Binary Tree ADT, the space complexity is of the $O(n)$ and time complexity is as given in Table 20.2

<i>Operation</i>	<i>Time</i>
<i>size, isEmpty</i>	$O(1)$
<i>iterator, positions</i>	$O(n)$
<i>hasNext, next (of returned iterators)</i>	$O(1)$
<i>replace</i>	$O(1)$
<i>root, parent, children, left, right, sibling</i>	$O(1)$
<i>hasLeft, hasRight, isInternal, isExternal, isRoot</i>	$O(1)$
<i>insertLeft, insertRight, attach, remove</i>	$O(1)$

Table 20.2 Time Complexity using Linked List Implementation

20.7 Binary Tree Traversal

In general processing a node of a binary tree means to perform some simple operation like printing the contents of the node or updating the contents of the node. Traversal in general means to process each node in the binary tree exactly once. Normally four types of traversal are defined in the case of binary trees. Binary tree traversals are special cases of general tree traversal.

1. Preorder Traversal -Each node is processed before any node in either of its two subtrees (Figure 20.8).
2. Inorder Traversal -Each node is processed after all nodes in its left subtree is processed but before any node in its right subtree is processed.
3. Postorder Traversal -Each node is processed after all nodes in both of its subtrees are processed,
4. Level order Traversal -Each node at level l is processed before any node at level $l+1$

20.7.1 Preorder Traversal

Now let us discuss the traversals one by one with illustrative example. The logic of the Preorder Traversal of Binary Tree is shown in Figure 20.8. The Figure also shows the recursive algorithm for Preorder Traversal. Figure 20.9 (a-e) illustrates the the preorder traversal of the tree given in the example.

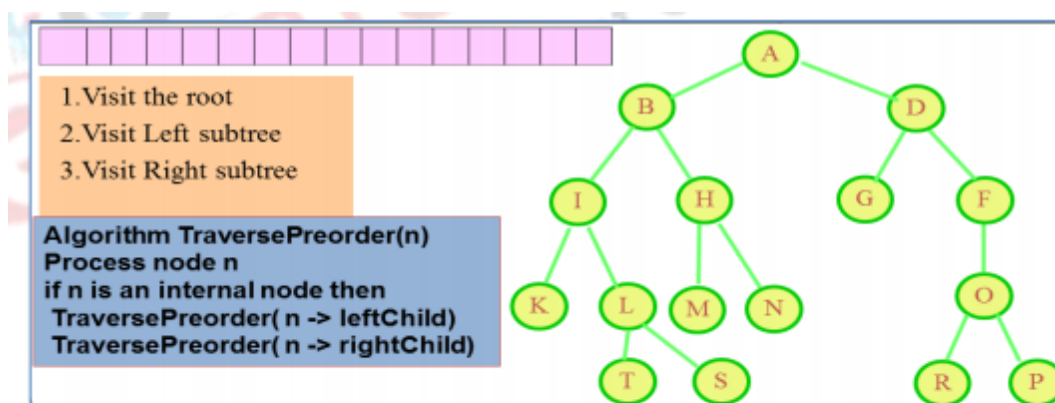
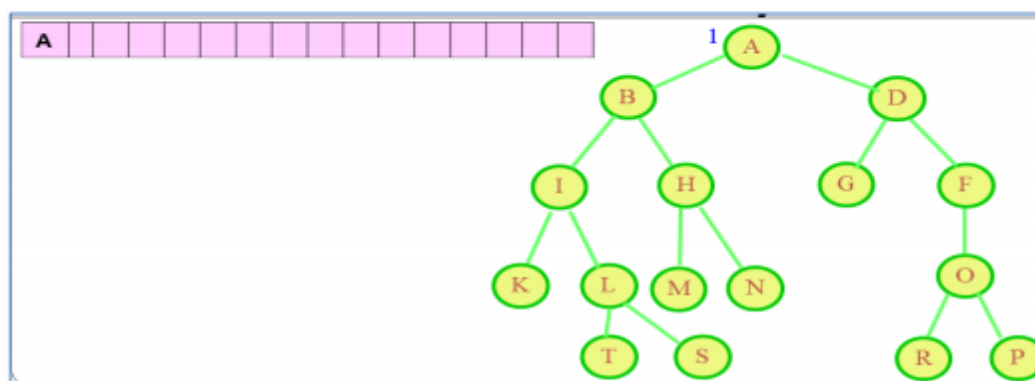
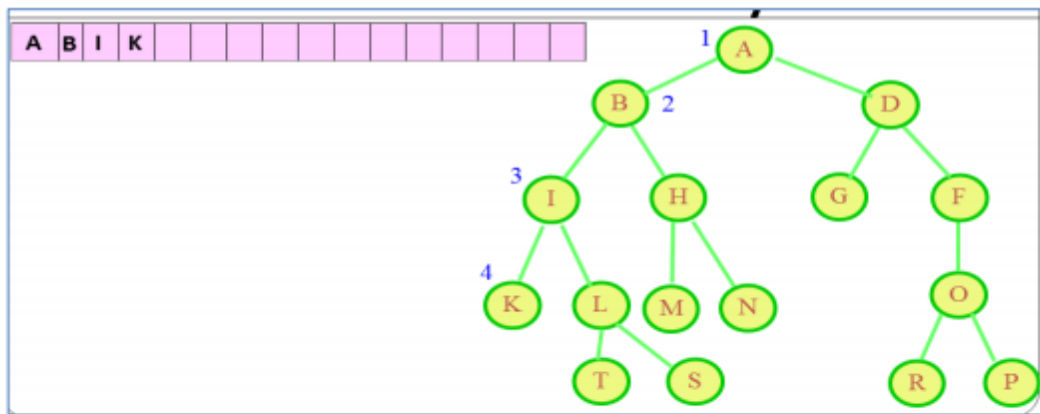


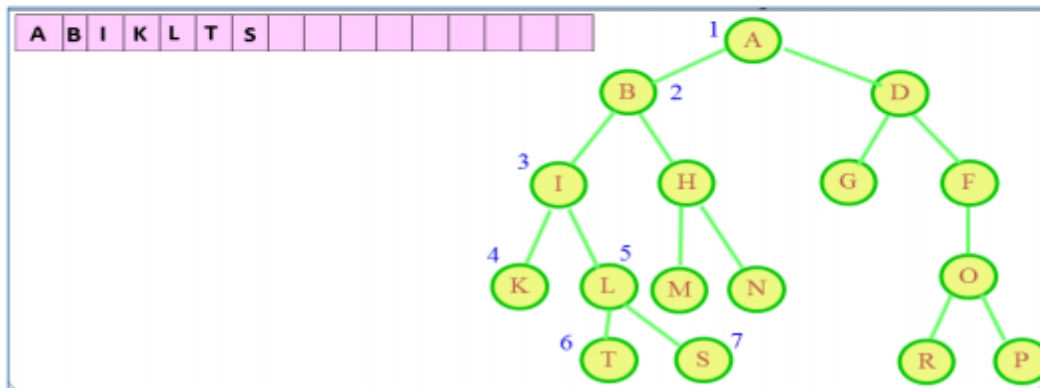
Figure 20.8 Preorder Traversal



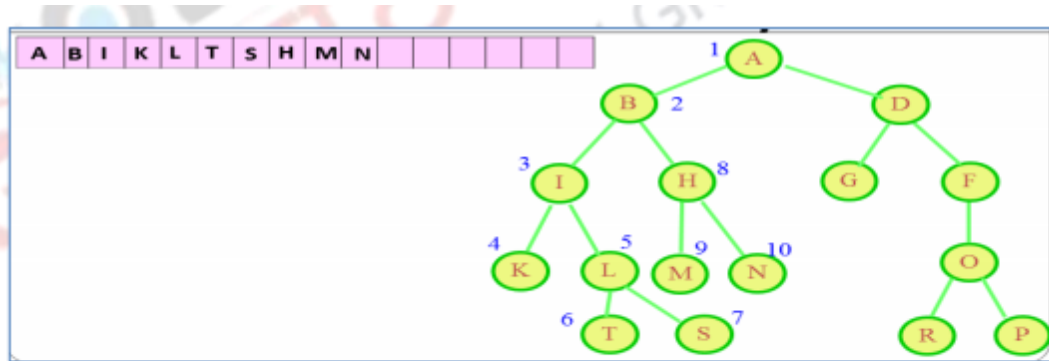
(a)



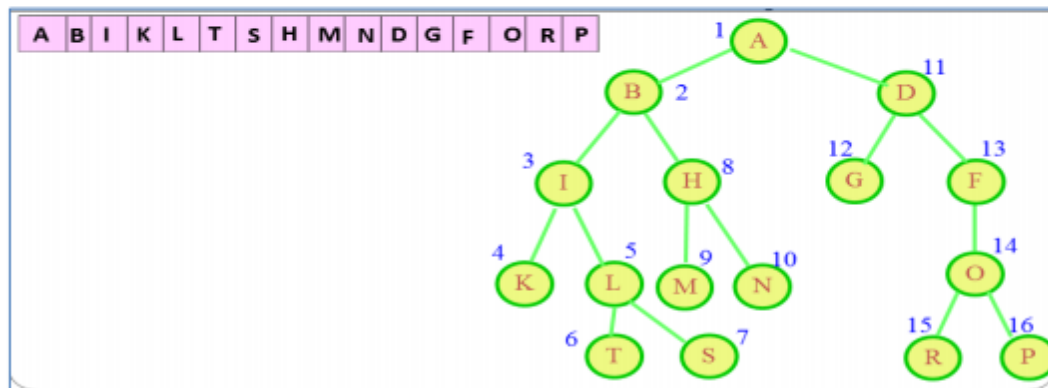
(b)



(c)



(d)



(e)

Figure 20.9 Illustrative Example of Preorder Traversal

Figure 20.9 (a) shows that in case of the preorder traversal, the root node A (1) is processed first. Figure 20.9 (b) shows B (2) the root of the left subtree of A being processed, followed by I (3) root of left subtree of B being processed, followed by K (4) the root of left subtree of I being processed. Now since K has no children, the processing of left subtree of I is completed. Figure 20.9 (c) shows L (5) the root of right subtree of K being processed, followed by the T (6) root of left subtree of L being processed. Now since T has no children, the processing of left subtree of L is completed and S (7) the right subtree of L is processed. This completes the processing of left subtree of B rooted at I. Now we need to process the right subtree of B. Figure 20.9 (d) shows the processing of the root H (8) of the right subtree of B, the left subtree of H rooted at M (9) and the right subtree of H rooted at N (10). Since M & N are leaves, this completes the processing of the complete left subtree B of the root A. Similarly the right subtree is processed and the complete preorder traversal is shown in Figure 20.9 (e).

20.7.2 Inorder Traversal

The logic of the Inorder Traversal of Binary Tree is shown in Figure 20.10. The Figure also shows the recursive algorithm for Inorder Traversal. Figure 20.11 (a-d) illustrates the inorder traversal of the tree given in the example.

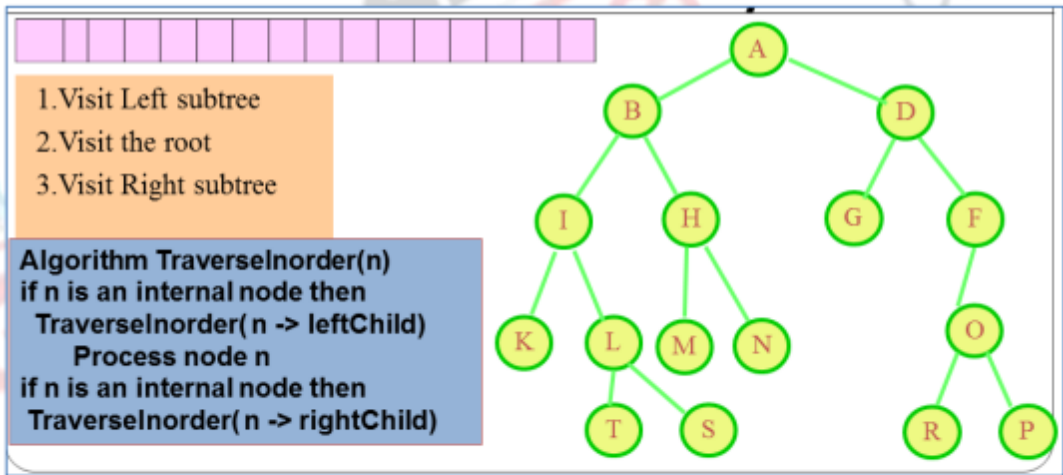
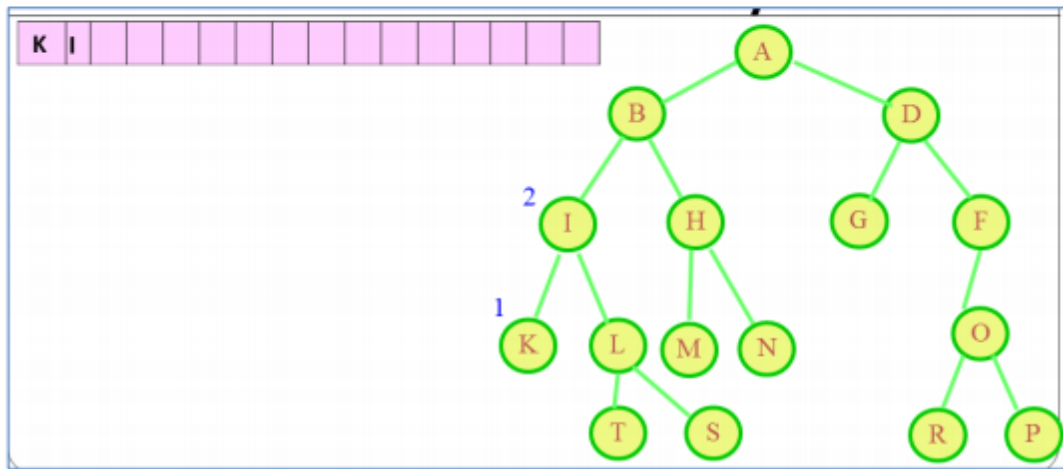
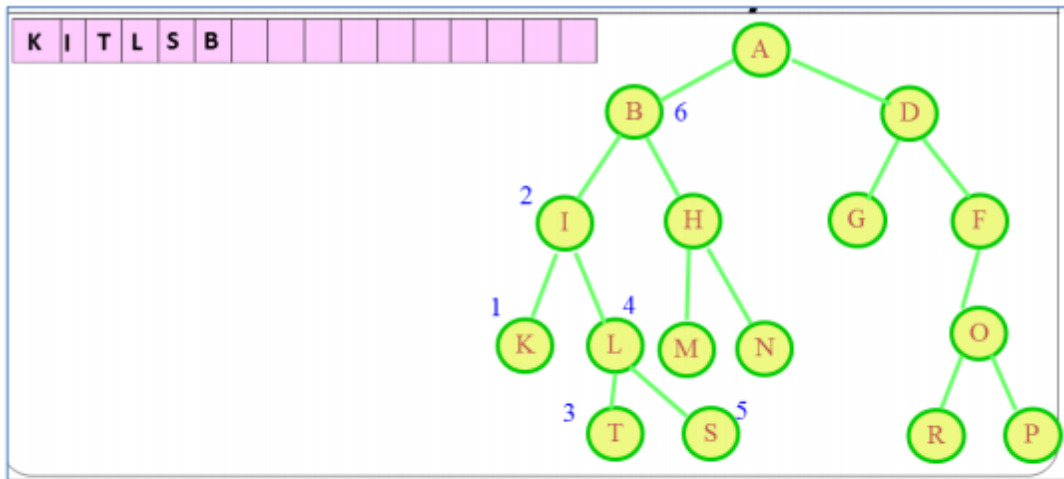


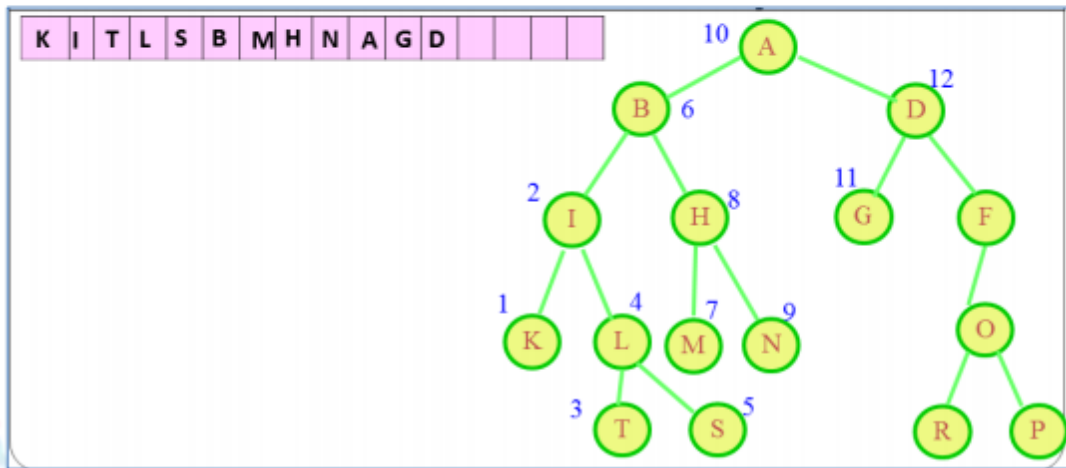
Figure 20.10 Inorder Traversal



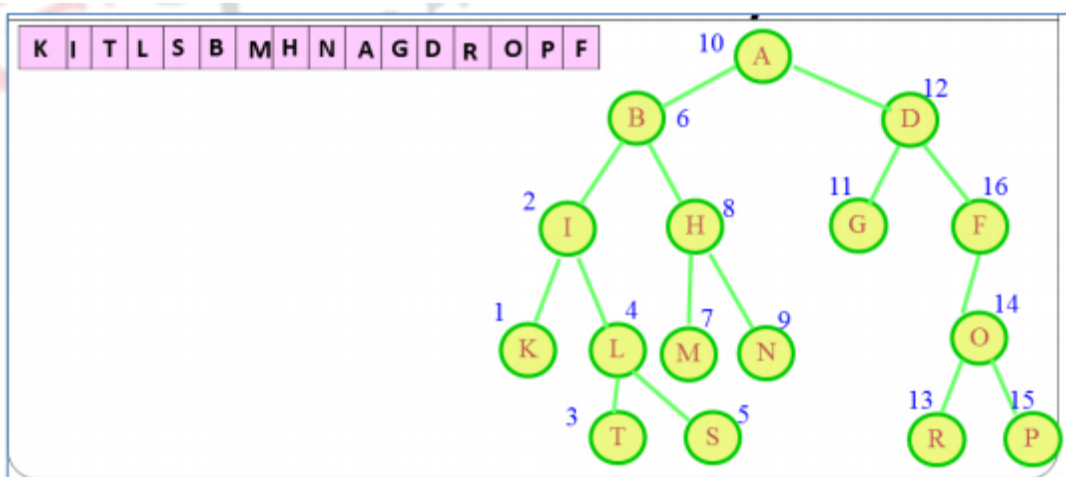
(a)



(b)



(c)



(d)

Figure 20.10 Inorder Traversal

Figure 20.11 (a) shows that in case of the inorder traversal, the leftmost node of the tree K (1) is processed first and then its parent I (2) is processed next. Figure 20.11(b) shows the right subtree of I rooted at L being processed where first root T (3) of left subtree of L is processed, followed by root L (4) being processed and then S (5) the root of right subtree of L is processed. Now the left subtree of B is completed so the root B (6) is processed. Figure 20.11 (c) shows the processing of right subtree of B rooted at H. Here first the root M (7) left subtree of H followed by H (8) and then root N (9) of right subtree of H are processed since M and N are leaves. Now the processing of the left subtree of A is completed, we need to process A (10). Now we process the right subtree of A rooted at D is to be processed. So we start by processing left subtree of D which has a leaf G (11). Similarly the right subtree is processed and the complete inorder traversal is shown in Figure 20.9 (d).

20.7.3 Postorder Traversal

The logic of the Postorder Traversal of Binary Tree is shown in Figure 20.12. The Figure also shows the recursive algorithm for Postorder Traversal. Figure 20.13 (a-d) illustrates the postorder traversal of the tree given in the example.

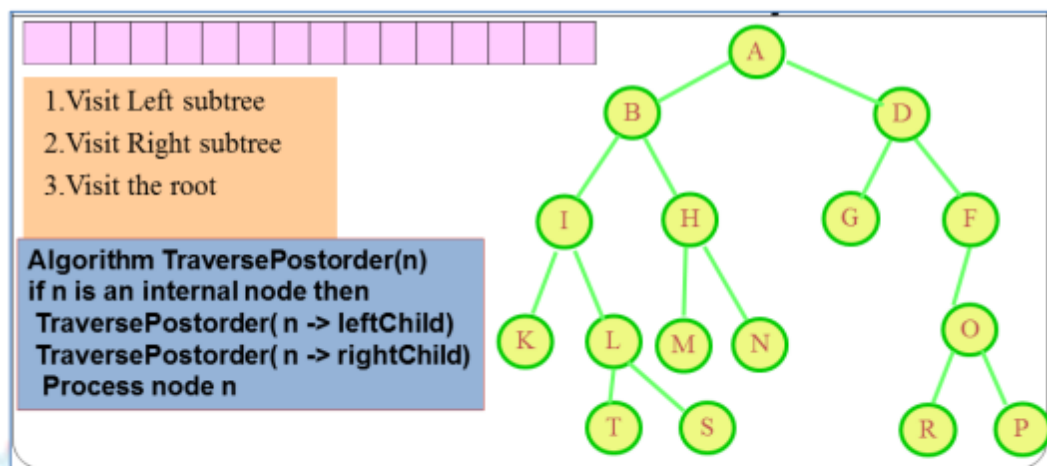
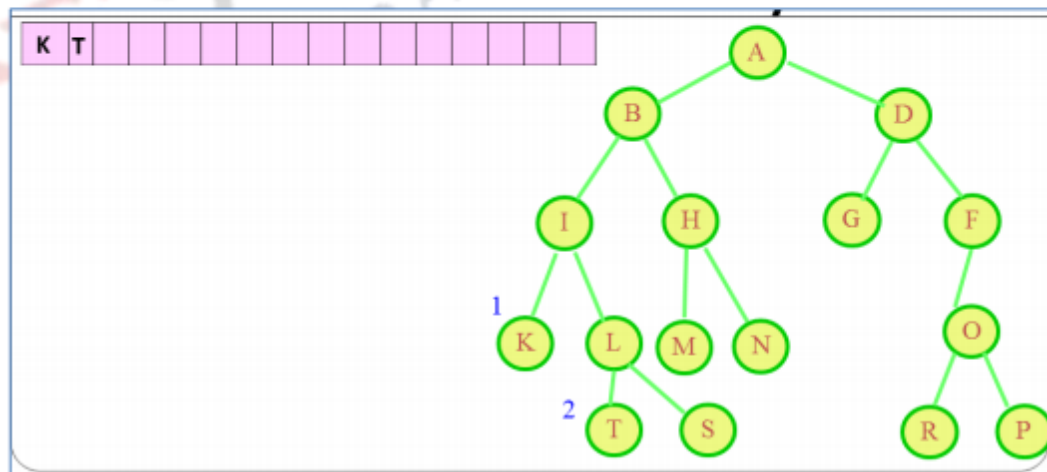
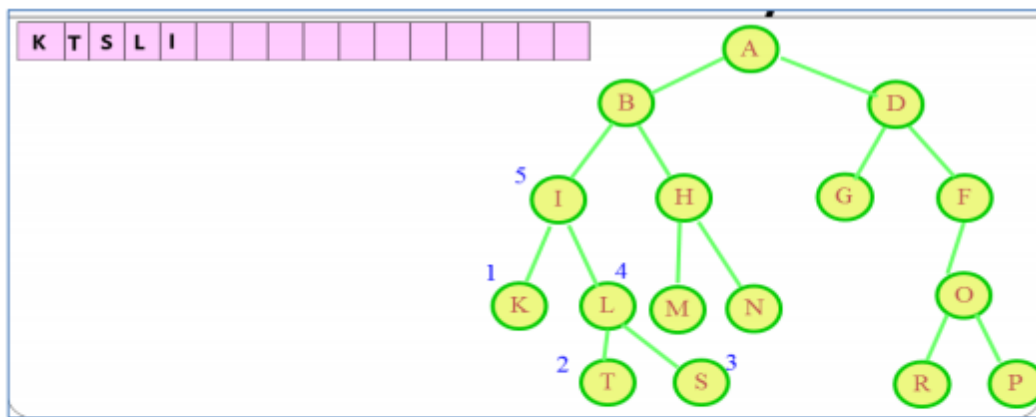


Figure 20.12 Postorder Traversal

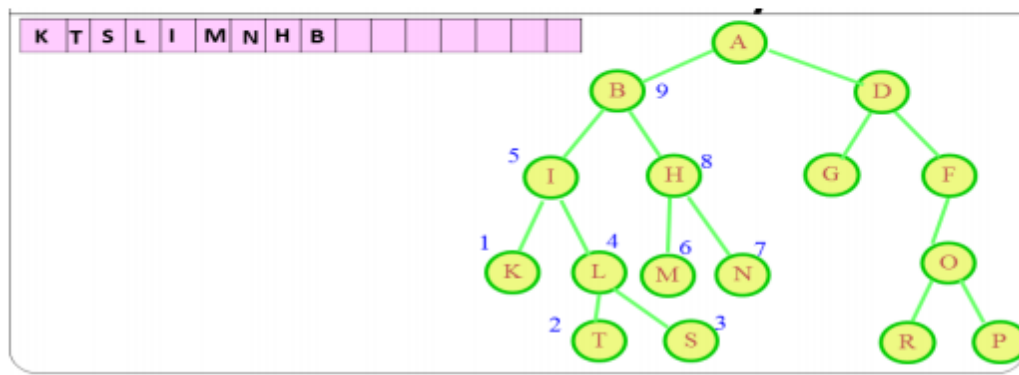


(a)

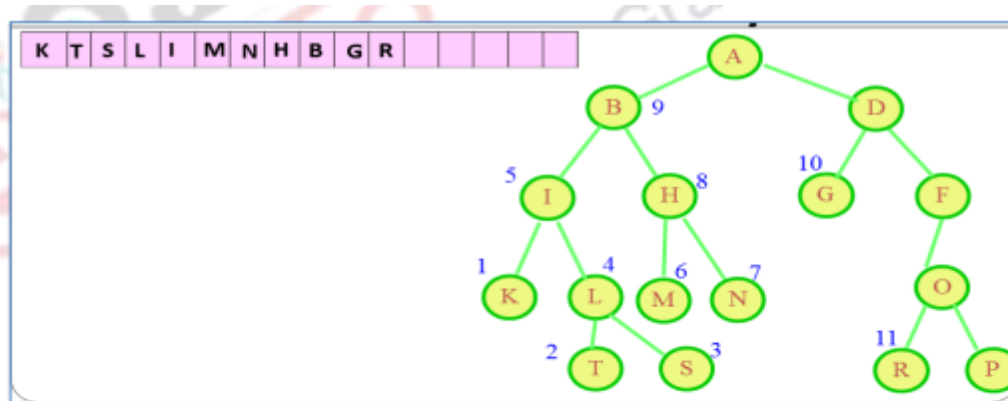
Figure 20.13 (a) shows that in case of the postorder traversal, the leftmost node of the tree K (1) is processed first and then the right subtree of I rooted at L needs to be processed. Here again the left subtree T (2) of L is first processed. Figure 20.13 (b) shows the right subtree S (3) of L is processed. Now since both left and right subtrees of L have been processed, L (4). Now since left and right subtrees of I have been processed, I (5) is processed.



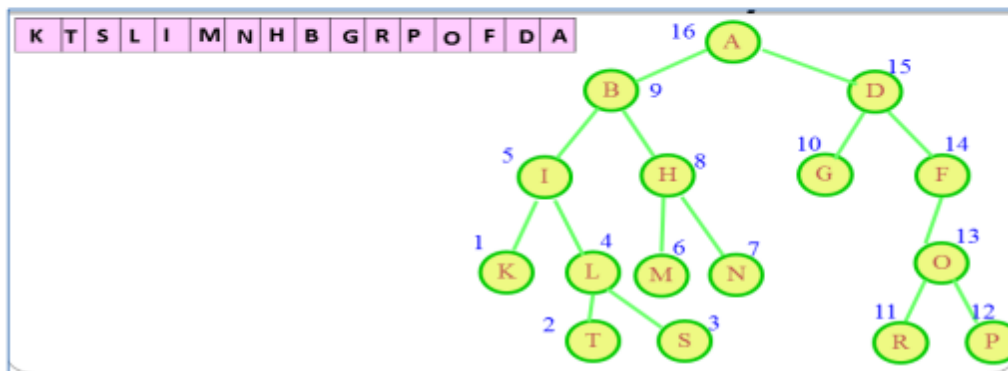
(b)



(c)



(d)



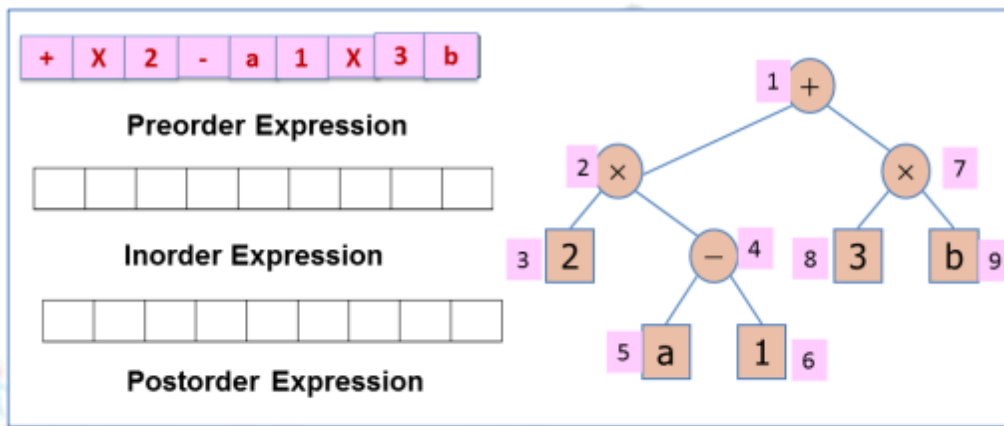
(e)

Figure 20.13 Illustrative Example of Postorder Traversal

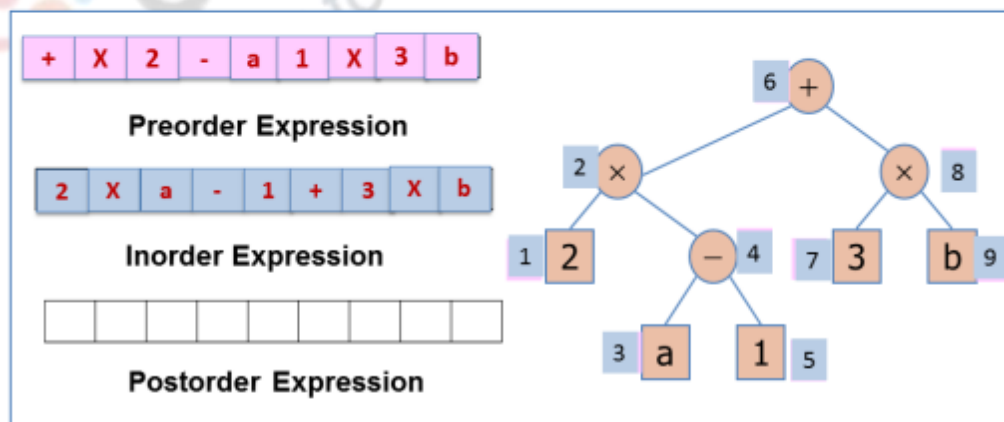
Figure 20.13 (c) shows the processing of right subtree of B rooted at H. Here first the root M (6) left subtree of H and then root N (7) of right subtree of H are processed since M and N are leaves, and then root H (8) is processed. Now the processing of the left and right subtree of B is completed, so we need to process B (9). Figure 20.13(d) shows the processing of the right subtree of A rooted at D. We start by processing left subtree of D which has a leaf G (10). Then we process the leftmost node R (11) of the right subtree of D. Similarly the right subtree is processed and the complete inorder traversal is shown in Figure 20.13 (e).

20.7.4 Expression Tree

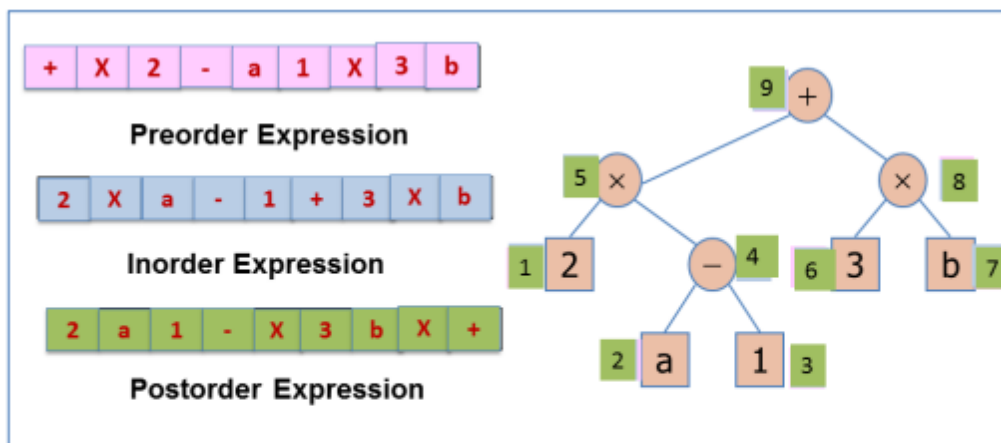
If we carry out the preorder, inorder, and postorder traversals of expression trees we can obtain the preorder, inorder and postorder expressions respectively. This is illustrated in Figure 20.14 (a-c).



(a)



(b)



(c)

Figure 20.14 Traversals of Expression Tree

20.7.5 Level Order Traversal of Binary Tree

The logic of the Level Order Traversal of Binary Tree is shown in Figure 20.15. Figure 20.16 (a-d) illustrates the level order traversal of the tree given in the example.

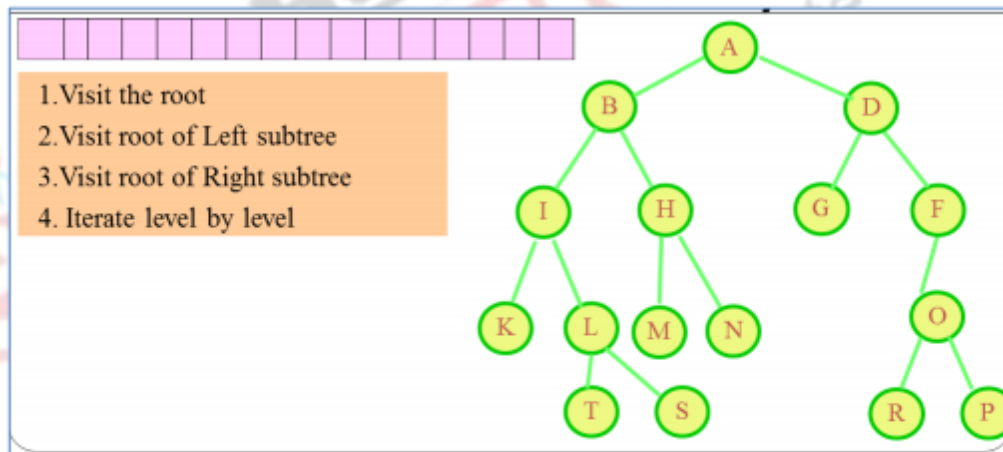
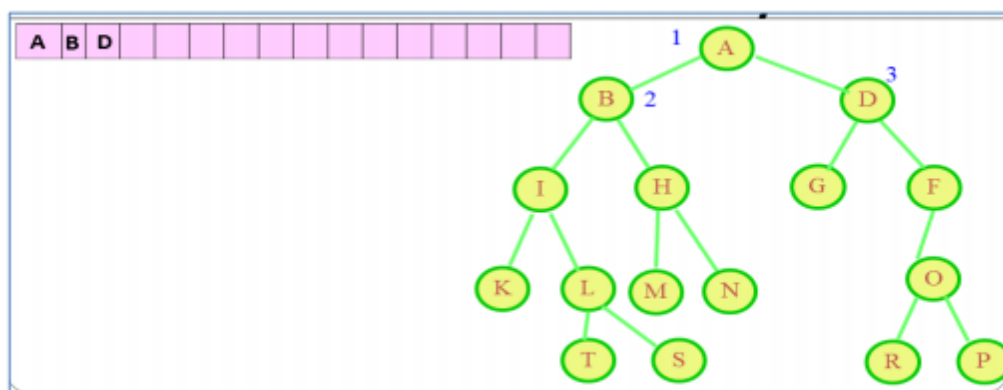
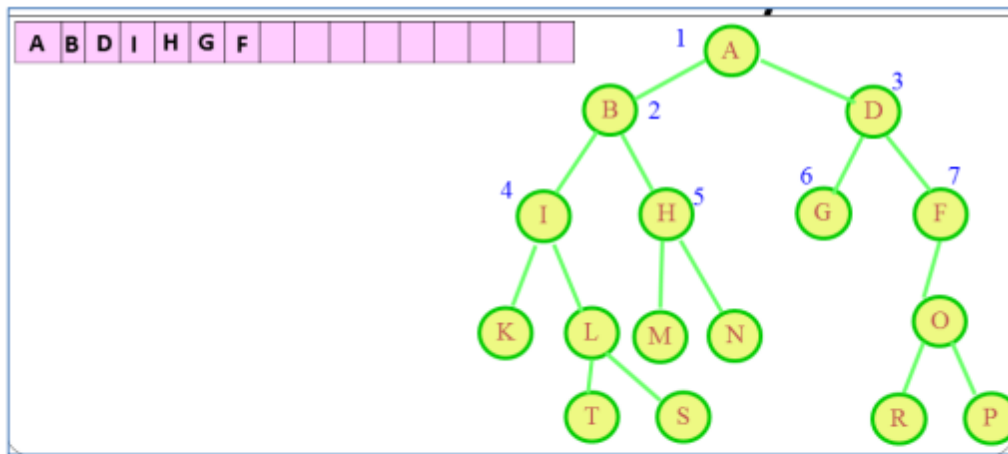


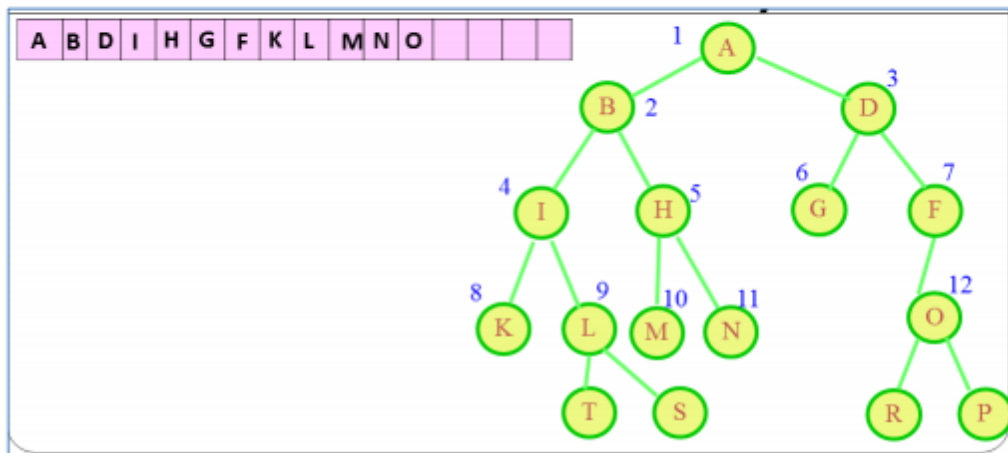
Figure 20.15 Level Order Traversal



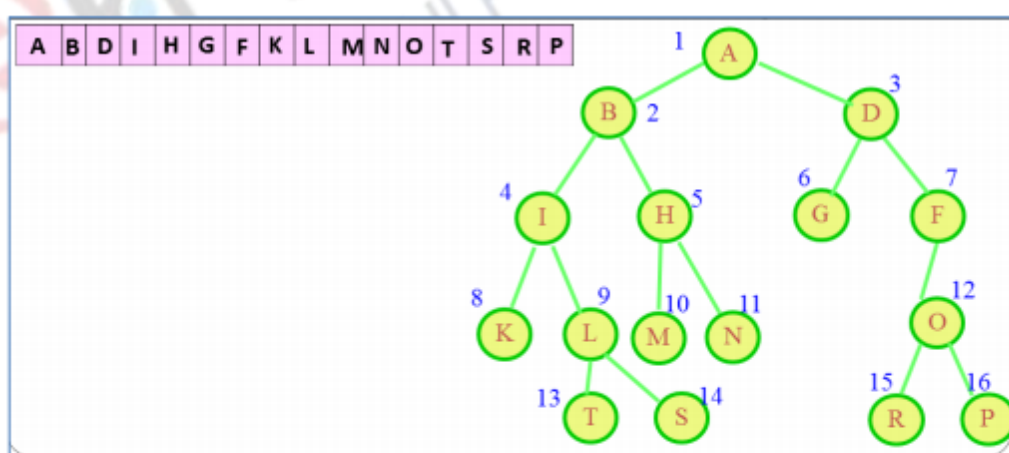
(a)



(b)



(c)



(d)

Figure 20.16 Illustrative Example of Level order Traversal

Figure 20.16 (a) shows the processing of root A (1) and of nodes B (2) and D (3) at

level 1. Figure 20.16 (b) shows the processing of nodes I (4), H (5), G (6) and F (7) at level 2. Figure 20.16 (c) shows the processing of nodes K (8), L (9), M (10), N (11) and O (12) at level 3. Finally figure 20.16 (d) shows the processing of nodes T(13), S (14), R (15) and P(16) at level 4.

20.8 Binary Tree ADT

The first operation associated with Binary Tree ADT is the empty tree operation The Boolean operation `EmptyTree()` which has no precondition, while the post-condition is that it returns `True` if Tree is empty and `false` otherwise Next we will discuss a generic traversal operation where the type of traversal is specified as a parameter. The operation `Traverse(Order Ord)` has the precondition: that the Binary Tree must not be empty. The post-condition is that the binary tree is traversed according to value of `ord`. We have if

- 1) `Ord =Preorder` – Traverses the tree using preorder traversal
- 2) `Ord =Inorder` – Traverses the tree using inorder traversal
- 3) `Ord =Postorder` – Traverses the tree using postorder traversal
- 4) `Ord =Level order` – Traverses the tree using level order traversal

Another important traversal of the binary tree is the Euler Tour traversal of Binary Tree where unlike other traversals a node can be visited more than once. This is a generic traversal of a binary tree and includes as special cases the preorder, postorder and inorder traversals. We walk around the tree and visit each node three times – on the left (preorder), from below (inorder) and on the right (postorder). All three tree traversals are unified by relaxing the requirement that each node be visited once. Applications of this traversal include calculating the number of descendants of a node, printing a fully parenthesized arithmetic expression, etc..

Web Links
www.cse.unt.edu/~huangyan/3110/Lectures/Trees.ppt
www2.latech.edu/~box/ds/chap8.ppt
www.eagle.tamut.edu/faculty/igor/my%20classes/cis-305/lecture-10.ppt
nlg.csie.ntu.edu.tw/courses/Datastructure/chapter5.ppt
http://www3.cs.stonybrook.edu/~jackie/
http://courses.cs.vt.edu/cs2604/spring04
http://ww0.java5.datastructures.net/hand
Supporting & Reference Materials
Michael T. Goodrich, Roberto Tamassia, Michael H. Goldwasser, "Data Structures and Algorithms in Java", Wiley; 6 edition, 2014
Carrano and Henry, "Data Structures and Problem Solving with C++: Walls and Mirrors", Pearson; 6 edition, 2012
Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Pearson; 3rd Edition, 2011
Robert Sedgewick, Kevin Wayne "Algorithms", Addison-Wesley Professional, 4th Edition, 2011
Thomas H. Cormen, et.al, "Introduction to Algorithms" The MIT Press; 3rd edition, 2009
Anany Levitin "Introduction to the Design and Analysis of Algorithms", Pearson; 3rd edition 2011